

Comunidade Conectada

Smart HAS — Mobile Hybrid App e a Sociedade 5.0

De um protótipo Flutter com dados mockados a um sistema full-stack real: app React Native, backend Spring Boot com Firebase e painel administrativo Angular — os três integrados pelo mesmo contrato de API.



React Native + Expo



Spring Boot + Firebase



Angular



Yuri Sousa

RM 556163 · FIAP 2026



SMART HAS

Comunidade Conectada

Colabore, compartilhe e ajude sua comunidade a crescer de forma sustentável e inteligente.

COMEÇAR AGORA

O que é o Smart HAS

Um app híbrido de economia colaborativa para a Sociedade 5.0



Comunidade conectada

- Pessoas oferecem e pedem recursos entre si
- Ferramentas, saúde, livros, alimentos
- Por empréstimo, troca ou doação



Geolocalização

- Recursos aparecem no mapa por proximidade
- Facilita encontros presenciais na vizinhança
- Mapa interativo real (OpenStreetMap)



ConectaIA

- Assistente de IA integrado ao app
- Ajuda a melhorar descrições de ofertas
- Tira dúvidas dos usuários em tempo real



Sociedade 5.0 na prática

Tecnologia usada para resolver um problema social concreto: aproximar vizinhos que têm recursos ociosos de quem precisa deles — com dados reais, não simulados.

O ponto de partida (Fase 4)

Um protótipo Flutter completo na interface — mas sem nenhum dado real por trás



O que já existia

- 11 telas funcionais: onboarding, login, home, mapa, chat, chatbot, oferta, perfil
- Arquitetura com Provider / ChangeNotifier
- Integração com IA (Gemini) para o chat
- UI consistente e responsiva



O que faltava

- Usuários, recursos e mensagens 100% mockados em memória
- Nada persistia — reiniciar o app apagava tudo
- Sem autenticação real, sem backend, sem banco de dados
- Botão "Publicar Oferta" nem sequer salvava o recurso



O desafio desta fase

Transformar a interface pronta em um sistema real — persistência, autenticação e uma API que o app e o painel administrativo pudessem consumir de verdade.

Escolha da stack mobile

React Native não foi escolha de gosto: foi o que alinha a entrega ao conteúdo ensinado na fase



React Native

ESCOLHIDO

- 5 dos 13 capítulos da fase são só sobre React Native
- Opção A do enunciado pede View / Text / Image / Button + React Navigation, ao pé da letra
- Node.js já disponível; Expo permite montar e testar rápido
- Hooks (useState / useEffect) e AsyncStorage seguindo os Cap. 05 e 06

VS



Flutter

VERSÃO ANTERIOR

- 11 telas da Fase 4, já integradas ao backend real
- Mantido no repositório como histórico do projeto
- Não exercita nenhum conteúdo específico desta fase
- Serve de base de referência para comparar a migração

5/13

capítulos da Fase 5 são dedicados a React Native

Migrar o app foi a forma de aplicar de fato o conteúdo da fase, em vez de apenas manter o que já funcionava em Flutter.

Roadmap tecnológico

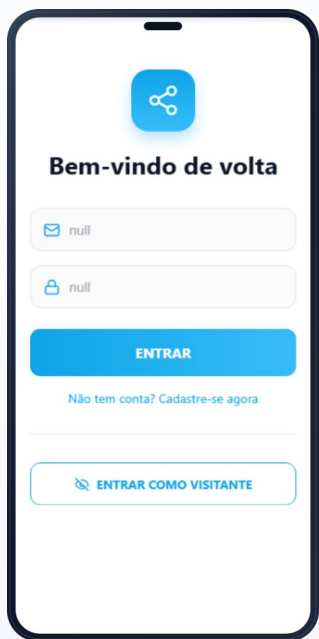
Onde o projeto estava, o que esta entrega resolve e o que fica para as próximas fases



O que muda nesta entrega: os dados deixam de ser simulados — cada tela do app passa a ler e gravar no Firestore através da API.

O app em React Native

As telas da Fase 4 reconstruídas com componentes nativos — rodando contra a API real



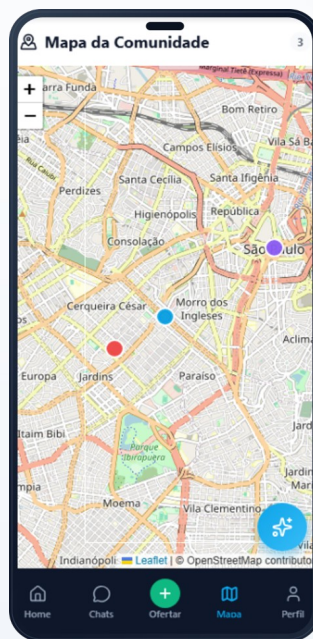
Login real

Firebase Auth



Home

recursos da API



Mapa

Leaflet + OSM



ConectaIA

Gemini

O que foi aplicado



Componentes nativos

View, Text, Image, Button e FlatList — Opção A do enunciado



React Navigation

Stack + Bottom Tabs entre as 8 telas do app



Hooks de estado

useState e useEffect consumindo a API (Cap. 05 e 06)



AsyncStorage

Token JWT guardado no dispositivo entre sessões

Backend Spring Boot

Java 21 · Spring Boot 4 · Firebase (Cap. 13) — uma API REST para o app e o painel



Firebase Auth

- Cadastro e login reais
- Senha nunca trafega pelo backend



Firestore

- Banco NoSQL na nuvem
- CRUD completo e persistente



3 entidades

- User, Resource e Message
- DTOs dedicados por operação



Camadas

- Controller → Service → Repository
- Injeção por construtor



Erros padronizados

- 400 / 401 / 403 / 404 / 409
- Formato único de resposta



Swagger / OpenAPI

- Documentação interativa
- Disponível em /swagger-ui

Todas as regras de negócio ficam no backend — app e painel apenas consomem a mesma API.

API REST — cobertura e verificação

Não é só código escrito: cada rota foi exercitada contra o Firebase real

18

endpoints REST



3

entidades persistidas



100%

testado ponta a ponta



Principais rotas

POST

/api/auth/register • /api/auth/login

GET PUT

/api/users/me • /api/users (ADMIN)

CRUD

/api/resources

GET POST

/api/messages • /api/messages/conversations

GET

/api/admin/stats (ADMIN)



Verificado de verdade



Servidor compilado e rodando localmente (./mvnw spring-boot:run)



Cadastro no Firebase Auth e login com as mesmas credenciais, em chamadas separadas



Regras de permissão dono-vs-admin conferidas na prática



Bug real no SDK do Firebase ("Not in GZIP format") encontrado e contornado

Dashboard administrativo Angular

O mesmo backend, agora consumido por um painel web — com os requisitos do enunciado aplicados



Rotas

/home pública · /login e /admin protegidas por AuthGuard



HttpClient + Interceptor

Anexa o token JWT automaticamente em toda chamada



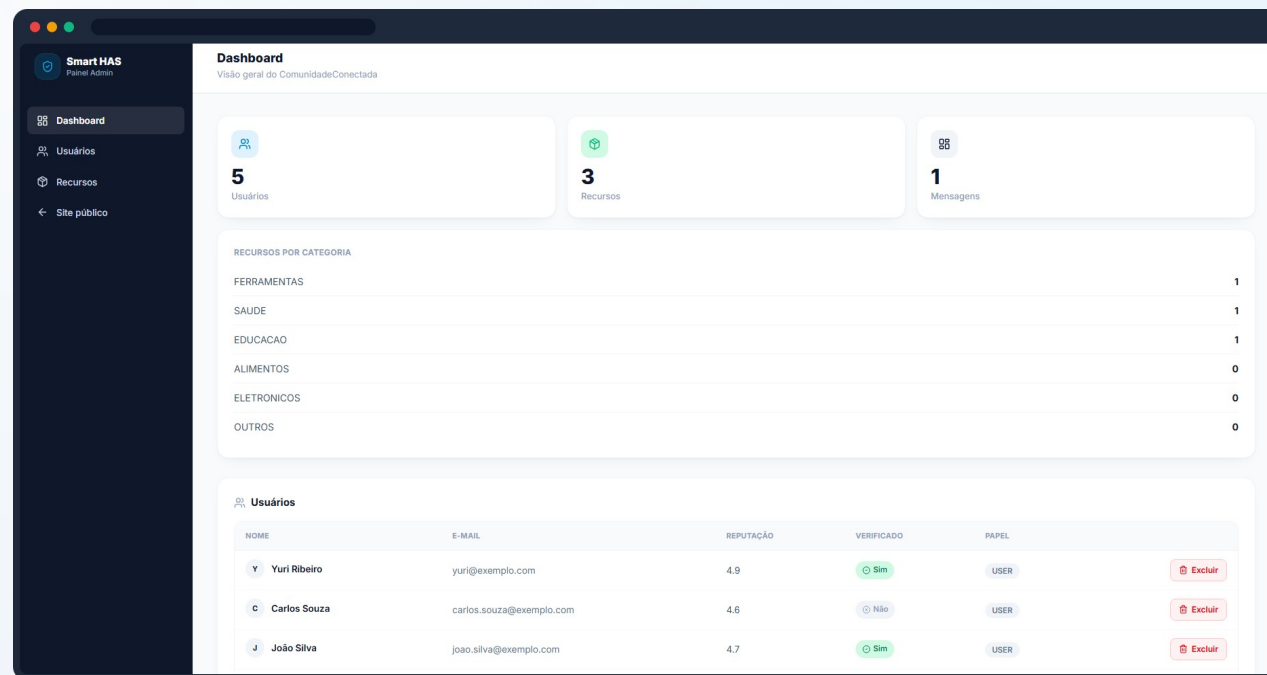
4 formas de binding

{{ }} interpolação · [] property · () evento · [()] ngModel



*ngIf / *ngFor

Estados de carregamento, erro e vazio nas listas



Print real de localhost:4200/admin — estatísticas, usuários e recursos vindos da API Spring Boot.

Integração ponta a ponta

As três aplicações compartilham o mesmo contrato de API e os mesmos dados reais



Evidência: um recurso renomeado via API apareceu, ao vivo, já atualizado no dashboard Angular.

CONCLUSÃO

De protótipo a sistema real

O Smart HAS deixou de ser uma interface com dados mockados e passou a ser um sistema full-stack: app React Native, backend Spring Boot com Firebase Authentication e Firestore, e dashboard Angular — os três integrados pelo mesmo contrato de API e alinhados ao conteúdo desta fase.



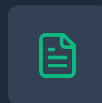
Código-fonte

[github.com/YuriRASousa/
ComunidadeConectada_FASE5](https://github.com/YuriRASousa/ComunidadeConectada_FASE5)



Vídeo de demonstração

youtu.be/6GpY25Bgr6E



Relatório completo

[Relatorio_SmarthAS_Fase5_Cap1.pdf](#)

Obrigado!



Yuri Sousa · RM 556163

FIAP · 2026